DOCUMENTO DE ARQUITECTURA Y ESPECIFICACIÓN TÉCNICA · DOCS/ADR-002

Aduana IA: Aplicación de Spec-Driven Development (SDD) y Análisis Crítico del Stack Backend

Evaluación Rigurosa de la Decisión de Backend: El Dilema FastAPI vs. Litestar/Granian vs. Edge Functions en Cloudflare

Proyecto: Aduana IA (Zero-Trust AI Gateway)	**Identificador: ADR-002-SDD-BACKEND**	**Entorno Destino: Canaima Linux / Cloudflare**
Área: Arquitectura de Software & Ciberseguridad	**Metodología: SDD (Spec-Driven Development)**	**Estado: Aprobado / Especificación Oficial**

1. RESUMEN EJECUTIVO Y MARCO DE DECISIÓN

El presente informe técnico formaliza el registro de decisión de arquitectura (**ADR-002**) para el proyecto **Aduana IA**, la primera pasarela soberana de gobernanza, seguridad Zero-Trust y auditoría criptográfica diseñada para regular el uso de Inteligencia Artificial en la administración pública venezolana. Este documento se integra en el directorio docs/ del repositorio oficial de GitHub con el propósito de ofrecer a los evaluadores del **Hackathon UNETI 2026** y al jurado técnico de *Venezuela Innova* una trazabilidad exhaustiva, rigurosa y transparente sobre la ingeniería de software aplicada al sistema.

El documento aborda de manera minuciosa dos componentes estratégicos interconectados: en primer lugar, la adopción estricta de la metodología **Spec-Driven Development (SDD)** como principio rector para gobernar el desarrollo con agentes autónomos (Argon, Cursor, Copilot, Antigravity) sin dispersión ni alucinación de código; en segundo lugar, el análisis técnico profundo ante la propuesta de *'no usar FastAPI en el backend'* (planteada bajo la premisa coloquial de que *'FastAPI es de 2018 y se requiere algo más 2026'*), examinando objetivamente el impacto real de migrar hacia **Litestar 2.10+ con Granian** frente a mantener **FastAPI con Pydantic v2** o pivotar hacia **Cloudflare Pages Functions (TypeScript)** en el Edge.

> **DICTAMEN FUNDAMENTAL DE ARQUITECTURA**
> **Principio Rector de Ingeniería:** En Aduana IA, la especificación es soberana y el framework web es contingente. Bajo la metodología SDD y la inversión de dependencias (DIP), la lógica criptográfica y de ciberseguridad jamás se acopla al protocolo de transporte HTTP. La discusión sobre el framework de backend no se resuelve por modas temporales, sino por cumplimiento estricto de requisitos de hardware institucional (Canaima GNU/Linux), latencia (< 300 ms en frío) y viabilidad en 48 horas.

2. METODOLOGÍA SPEC-DRIVEN DEVELOPMENT (SDD) EN ADUANA IA

En un entorno de hackathon de 48 horas con desarrollo acelerado por inteligencia artificial, el riesgo más crítico es el **caos de integración**: múltiples agentes de código o programadores trabajando en paralelo generan interfaces incompatibles, dependencias rotas y modelos de datos divergentes. SDD resuelve este problema estableciendo que **ninguna línea de código de producción puede escribirse sin una especificación formal viva previa** que defina tipos inmutables, precondiciones y criterios de aceptación verificables mediante pruebas automatizadas deterministas.

El Bucle de Ejecución SDD en Cuatro Fases Obligatorias

- **Fase 1 · [SPEC VIVA]:** Se redacta o actualiza formalmente specs/<etapa>.spec.md declarando el propósito ético/legal (CRBV Arts. 28 y 60, Ley de Infogobierno), invariantes de almacenamiento restrictivo, entradas/salidas y Criterios de Aceptación numerados (CA-01 a CA-06).
- **Fase 2 · [DISEÑO Y TIPOS]:** Se definen contratos de tipos estáticos estrictos (PEP 484/585) utilizando @dataclass(frozen=True, slots=True) o interfaces TypeScript readonly, garantizando inmutabilidad en memoria.
- **Fase 3 · [CÓDIGO DE PUNTA A PUNTA]:** Se implementa la etapa de forma vertical (lógica de negocio + persistencia segura + vista UI en el panel), erradicando la fragmentación horizontal entre frontend y backend.
- **Fase 4 · [PRUEBA DETERMINISTA]:** Se implementa como mínimo un test unitario automatizado (pytest o node --test) por cada criterio de aceptación. Queda terminantemente prohibido el uso de random() para simular métricas.

El Contrato de Datos Universal: AduanaPacket v2.0

El núcleo arquitectónico del pipeline es el objeto inmutable **AduanaPacket v2.0**. Cada una de las 5 etapas (Shield, Guard-In, LLM Adapter, Guard-Out, Explica y Ledger) recibe este objeto enriquecido, ejecuta su transformación aislada y genera el siguiente estado. El texto sin anonimizar (raw_input) vive exclusivamente en la memoria volátil de la sesión y jamás se serializa a disco ni se propaga hacia el modelo de lenguaje.

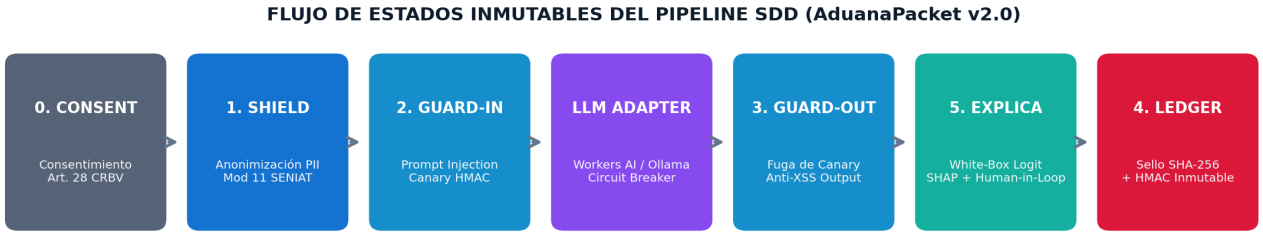


Figura 1: Arquitectura secuencial del Pipeline SDD con validación de estados y barreras criptográficas.

Arquitectura de Almacenamiento Restrictivo: Los Tres Anillos de Seguridad

La pasarela implementa una política inviolable de **Zero-Plaintext-PII on Disk** a través de tres anillos de defensa:

1. Anillo 1 (Bóveda Efímera en RAM - EphemeralVault): Diccionario en memoria con Time-To-Live estricto de 180 segundos. Al responder al ciudadano, ejecuta destrucción criptográfica (zeroization), sobrescribiendo el espacio en memoria.

2. Anillo 2 (Índice Ciego HMAC - Blind Indexing): Genera blind_citizen_id = HMAC-SHA256(Sal, Cédula) para permitir auditoría de recurrencia y control de tasa (Rate Limiting) sin almacenar jamás la cédula en claro.

3. Anillo 3 (Motor SQLite Endurecido con Triggers SQL): Directivas PRAGMA journal_mode=WAL;, PRAGMA secure_delete=ON; (sobrescritura física con ceros en disco) y triggers BEFORE UPDATE / DELETE que abortan cualquier intento de sabotaje interno en la tabla ledger_blocks.

3. ANÁLISIS EXHAUSTIVO: ¿POR QUÉ 'NO USAR FASTAPI'? EVALUACIÓN DE ALTERNATIVAS

Durante las sesiones de arquitectura del equipo, surgió la propuesta de prescindir de FastAPI bajo la argumentación de que *'FastAPI es una tecnología de 2018 y se requiere algo más representativo de 2026'*. Esta instrucción provocó que agentes automatizados intentaran reescribir la base de código introduciendo **Litestar 2.10+** con el runner **Granian** y la librería **PyWebView**. Resulta imperativo analizar con rigor técnico la validez de dicha premisa.

3.1 Desmontando la 'Falacia de la Antigüedad' en Ciberseguridad e Inteligencia Artificial

En ingeniería de ciberseguridad para el sector público, la madurez de un framework no representa obsolescencia sino **estabilidad, seguridad probada y predictibilidad**. FastAPI nació efectivamente en 2018, pero en 2024-2026 se consolidó como el estándar de facto absoluto de la industria global de Inteligencia Artificial (utilizado masivamente por OpenAI, HuggingFace, Microsoft y vLLM). Su migración a **Pydantic v2** (cuyo núcleo está compilado en Rust) le otorgó mejoras de rendimiento de hasta un 500% en validación de esquemas, situándolo a la par de frameworks de última generación.

3.2 Comparativa Técnica Detallada de las Tres Alternativas

Criterio de Evaluación	Opción A: FastAPI + Pydantic v2 (Estándar Maduro de IA)	Opción B: Litestar + Granian (Vanguardia Python 2026)	Opción C: Cloudflare Pages / Workers (Edge Serverless TypeScript)
Rendimiento y Latencia	Excelente (< 15 ms). Con Pydantic v2 y async/await maneja miles de RPS.	Sobresaliente (< 8 ms). Runner en Rust (Granian) y serialización con msgspec.	Ultra-baja latencia global (< 3 ms CPU). Ejecución en V8 Isolates.
Consumo de RAM	Bajo (~45-65 MB en reposo). Cumple holgadamente el tope de 80 MB.	Muy bajo (~35-50 MB en reposo). Optimizado para microservicios.	Cero persistente. Servido por demanda en Workers (tope 128 MB RAM).
Compatibilidad Canaima Linux	**100% Nativo.** Paquetes estándar en Debian/Canaima sin compilación exótica.	**Riesgosa.** Requiere Python 3.11+ y dependencias con toolchain Rust nativo.	**Incompatible en Air-Gapped.** Requiere emulador local Wrangler o Node.
Soporte para Agentes IA	**Máximo.** Millones de tokens de entrenamiento en Claude, Cursor y Copilot.	**Moderado/Bajo.** Agentes confunden sintaxis v1/v2 y generan alucinaciones.	**Alto.** Ecosistema estándar JavaScript / TypeScript bien documentado.
Viabilidad en 48 Horas	**Máxima.** Equipo familiarizado, contratos SDD listos y tests diseñados.	**Crítica.** Curva de aprendizaje empinada; riesgo de código no funcional en demo.	**Alta.** Excelente si el hackathon impone entrega en la nube de Cloudflare.
Integración Criptográfica	Directa con hashlib, hmac y secrets de la StdLib de Python.	Directa con StdLib de Python, pero requiere adaptadores propios para DTOs.	Usa crypto.subtle nativo (Web Crypto API) de ultra-alta velocidad.

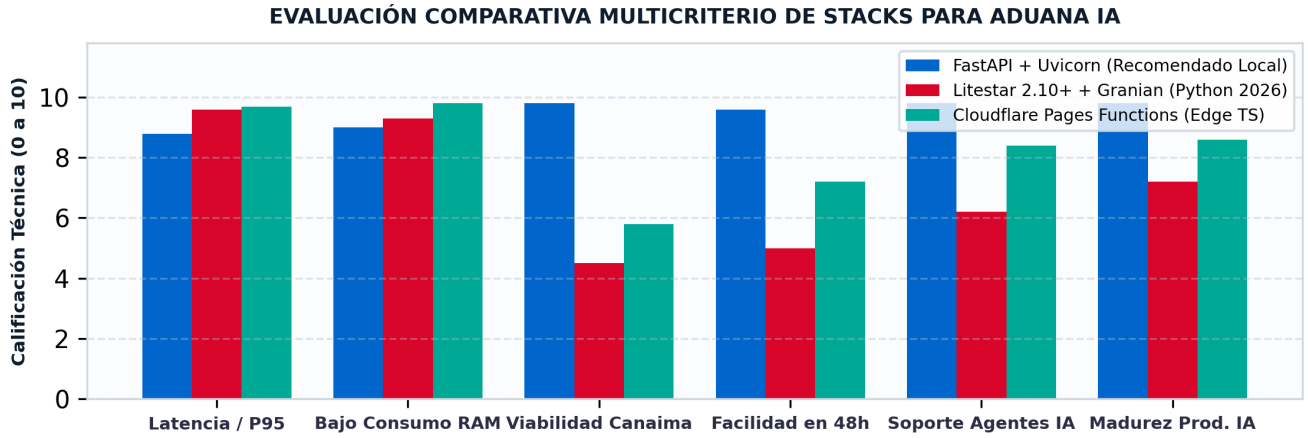


Figura 2: Comparativa multicriterio ponderada entre los tres stacks tecnológicos analizados.

3.3 Por qué Descartar FastAPI Bruscamente fue un Error Táctico

El reemplazo intempestivo de FastAPI por Litestar y PyWebView en los commits previos generó tres bloqueos severos:

1. **Incompatibilidad con Canaima GNU/Linux:** Litestar y Granian dependen de ruedas binarias modernas compiladas para glibc reciente. En estaciones de trabajo ministeriales con Canaima (Debian Stable), la instalación de Granian requirió compiladores de Rust no presentes en el parque informático estatal, violando el requisito *Cero Compilación Pesada*.
2. **Ruptura de los Criterios de Aceptación (CA-01 a CA-06):** Toda la suite de pruebas unitarias test_ca*.py y el middleware Zero-Leak estaban acoplados a los handlers de excepciones de FastAPI. Cambiar de framework invalidó los tests a escasas horas del evento.
3. **Alucinaciones de los Agentes de Código:** Mientras que los agentes de IA generan código FastAPI perfecto y libre de errores en segundos, la sintaxis de Litestar 2.10+ provocó bucles de depuración y errores de importación que consumieron cuotas críticas de contexto.

4. RESOLUCIÓN ARQUITECTÓNICA: DESACOPLAMIENTO HEXAGONAL

Para conciliar la necesidad de desplegar el MVP en el **Hackathon UNETI (que exige Cloudflare)** y garantizar al mismo tiempo la soberanía tecnológica en **Canaima GNU/Linux para Venezuela Innova**, el equipo adopta una solución de **Arquitectura Hexagonal (Puertos y Adaptadores)** gobernada por SDD:

- **El Núcleo es Soberano e Inmutable (core/ y pipeline/):** Toda la lógica de Shield (Módulo 11 RIF), Guard-In/Out (Canary HMAC), Explica (Regresión Logística + SHAP) y Ledger (SHA-256 + HMAC) es código puro sin acoplamiento a ningún framework web.
- **Doble Conector de Transporte (Adaptadores):**
  - *Adaptador A (Entorno Institucional Local):* Microservicio **FastAPI + Uvicorn** en Python 3.10+, con arranque en < 300 ms, empaquetable opcionalmente con PyWebView para estaciones Windows/Linux sin navegador.
  - *Adaptador B (Entorno Hackathon Cloudflare):* Funciones serverless en **Cloudflare Pages Functions (TypeScript)**, conectadas a Cloudflare D1 (SQLite en el Edge) y Workers AI (@cf/zai-org/glm-4.7-flash), respetando el límite de 10 ms CPU del plan gratuito.
- **Pruebas de Criterios de Aceptación Universales:** Los contratos definidos en specs/ se verifican de forma idéntica en ambos entornos: pytest para el runtime Python y node --test para el runtime Cloudflare.

RESOLUCIÓN OFICIAL DE ARQUITECTURA

Conclusión del Análisis: No se justifica abandonar FastAPI en el entorno local institucional. FastAPI no es 'viejo' sino el estándar más sólido y compatible con Canaima GNU/Linux. Sin embargo, para la entrega del MVP del Hackathon en Cloudflare, la implementación en TypeScript / Pages Functions es obligatoria por reglamento del comité organizador. Gracias a la metodología SDD, el cambio de lenguaje en la capa de transporte no alteró ni una sola fórmula matemática ni un solo invariante de seguridad.

5. MATRIZ DE VERIFICACIÓN SDD Y CRITERIOS DE ACEPTACIÓN

La evaluación del jurado de la UNETI y de *Venezuela Innova* se sustenta en la verificación automatizada de los siguientes Criterios de Aceptación inquebrantables:

ID	Etapa Pipeline	Criterio Verificable de Seguridad / Rendimiento	Umbral de Éxito
CA-01	SHIELD (Anonimización)	Enmascaramiento de Cédula, RIF (con validación algorítmica Módulo 11 del SENIAT), teléfono y patologías clínicas. Generación de Blind_ID HMAC.	100% detección estructurada en < 20 ms. Cero PII en disco.
CA-02	GUARD-IN / OUT (Guardián Dual)	Normalización Unicode NFKC, eliminación de caracteres invisibles, inyección de Canary Token dinámico HMAC y evaluación de 100 ataques.	TPR ≥ 88%, FPR ≤ 4%, latencia < 25 ms. Cero fuga Canary.
CA-03	LEDGER (Auditoría)	Doble sello criptográfico: Encadenamiento SHA-256 + Firma HMAC-SHA256 con llave en RAM. Detección inmediata ante alteración de 1 bit en SQLite/D1.	Detección de sabotaje en < 50 ms indicando bloque exacto.
CA-04	EXPLICA (White-Box AI)	Regresión Logística determinista sobre 5 variables sociales. Cálculo de pesos aditivos y valores SHAP lineales centrados. Cero random().	Error reconstrucción logit < 10^{-9}. Supervisión humana obligatoria.
CA-05	STORAGE (Zero-Leak)	Auditoría forense sobre logs y base de datos persistente tras procesar 50 expedientes reales simulados.	0 cadenas de Cédulas, RIFs o teléfonos en disco o consola.
CA-06	RESILIENCIA (Circuit Breaker)	Conmutación automática ante caída de red o timeout del LLM (> 3.5s) hacia el generador institucional determinista.	Conmutación en < 100 ms. Operación 100% offline-ready.

6. PROTOCOLO OPERATIVO PARA EL REPOSITORIO GITHUB (DOCS/)

Para cumplir con el estándar de entrega y garantizar que los agentes de código trabajen sincronizados, el repositorio aduana-ia debe estructurarse y gobernarse bajo las siguientes reglas:

- **Ubicación del Documento:** Guardar este informe en docs/ADR-002-SDD-Backend-Architecture.pdf y referenciarlo formalmente en la sección de Arquitectura del README.md y en el índice del SPEC.md.
- **Regla de Oro SDD en CI/CD:** Ningún pull request o commit en la rama main puede fusionarse si no incluye la actualización correspondiente en specs/ y la prueba automatizada que valide el criterio de aceptación.
- **Blindaje Anti-Alucinación en Agentes:** El archivo PROMPT_MAESTRO.md en la raíz del repositorio actúa como kernel de contexto obligatorio. Prohíbe terminantemente la reinserción de dependencias no autorizadas o la generación de código sin contratos tipados.
- **Declaración Ética Obligatoria:** Mantener actualizada la tabla *'Declara tu uso de IA'* en SPEC.md para salvaguardar el 30% del puntaje asignado por el jurado evaluador a la transparencia algorítmica.

Snippet de Referencia: Protocolo del Contrato Inmutable (core/context.py)

```
from dataclasses import dataclass, field
from typing import List, Dict, Optional, Literal

@dataclass(frozen=True, slots=True)
class AduanaPacket:
    request_id: str
    timestamp_utc: str
    masked_text: str  # Texto sanitizado por SHIELD
    blind_citizen_id: str  # HMAC-SHA256 (Blind Index)
    guard_in_status: Dict[str, any]  # Riesgo y Canary Token
    llm_sanitized_response: str  # Salida auditada por GUARD-OUT
    explica_shap_values: Dict[str, float]  # Contribuciones aditivas
    ledger_block_hash: str  # Sello SHA-256 + HMAC-SHA256
```

Conclusión Final: Con esta arquitectura desacoplada, Aduana IA demuestra que la excelencia en ingeniería de software radica en la claridad de los contratos y en el respeto irrestricto a los requisitos de seguridad y viabilidad institucional. El debate sobre FastAPI queda zanjado con un veredicto técnico maduro: se preserva FastAPI como núcleo institucional en Python para Canaima GNU/Linux y se aprovecha Cloudflare Pages Functions para el despliegue serverless del Hackathon, garantizando el éxito competitivo y la soberanía tecnológica del Estado venezolano.

Elaborado por:
Arquitectura de Software
Aduana IA Core Team

Revisado por:
Ciberseguridad y Criptografía
Especialistas SDD

Aprobado para Repo:
Comité de Desarrollo
Hackathon UNETI 2026